

Code-Layout, Readability and Reusability

Date	08 July 2026
Team ID	XXXXXX
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code.	Yes	Standard PEP 8 spacing maintained uniformly.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear division between logic, styles, and endpoints.
3	Meaningful Variable Names	Variables reflect their purpose clearly.	Yes	Descriptive, self-explanatory variable naming.
4	Function / Method Names	Functions are descriptively named.	Yes	Highly explicit function naming style.
5	Code Comments	Inline and block comments explain logic.	Yes	Well-segmented layout via block labels.
6	Modular Design	Code is split into reusable functions/modules.	Yes	Separate dedicated scripts for each AI feature.
7	No Redundant Code	Duplicate or unused code is removed.	Yes	Central router cleanly prevents duplicate logic.
8	Error Handling	Exceptions and errors are handled gracefully.	Yes	Structured try-except blocks wrap API calls.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	config.py (API configuration Setup)	Python / google-generativeai	Initialized broadly across all individual LLM modules.	High
2	render_feature_card()	Python / Streamlit Layout Engine	Reused dynamically 5 separate times on the main frontend Dashboard space.	High
3	clean_json_block()	Python / Regular Expressions	Can be integrated across any textual parsing layer processing structured JSON strings.	High
4	result_card()	Python / Custom HTML Wrapper	Reused repeatedly throughout different frontend module navigation tabs to show output blocks cleanly.	Medium
5	style.css	CSS / Streamlit Markdown Injector	Injected directly via load_css() to structure every distinct app page view uniformly.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean layout with a strong separation of concerns.
Readability	5	Intuitive code flow, clear logic, and neat formatting.
Reusability	5	High modularity makes it easy to port components elsewhere.
Documentation / Comments	4.5	Good sectioning; could use a few inline docstrings.
Overall Score	4.9	Excellent, production-grade application architecture.